

The system can investigate. It cannot act without you.

CoreOps is an AI operations command center for SQL Server, SSIS, SSRS, and Windows Server infrastructure — it watches for incidents, explains root cause with Claude, and proposes fixes. Every proposal stops at a real, enforced human-approval gate before anything touches production.

210 tests, all passing · **Live-verified**, not just unit-tested · Real Azure SQL, Blob Storage, Claude API · Zero fabricated data — **honest fallback** when a source is down

THE PROBLEM

Root-causing a SQL Server incident means manual cross-referencing, hours after the fact.

A job fails, or a report comes back empty. Someone has to correlate job logs, blocking sessions, and downstream reports across a dozen tools to explain why — usually well after it mattered. CoreOps watches continuously, correlates related signals, and asks Claude to explain root cause from live evidence, not a static runbook.

HOW IT WORKS

Detection to decision, in one traceable chain

Every step below shares one correlation ID — the whole story for a single incident is reconstructable after the fact, not just each step in isolation.

01 Continuous monitoring detects it

Polls SQL Server's live session state (`sys.dm_exec_requests`) on an interval. A genuinely blocked session — not noise — triggers the chain.

02 Claude explains root cause

A real model call over the actual evidence rows, not a canned response — returns a root-cause explanation and a confidence score.

03 Low confidence escalates

Below 60%, the system notifies the on-call operator by real push delivery rather than presenting a guess as settled.

04 A remediation is proposed — and blocked

The guardrail checks it's evidence-linked, of an allowed reversible type, and human-approved. Missing approval fails it, every time, no exceptions.

05 A signed-in operator decides

Approve executes the one real reversible action wired up; reject leaves everything untouched. Both are logged under the same identity that logged in.

REAL AUDIT TRAIL — ONE CORRELATION ID, RECONSTRUCTED

Actor	Event	Time
recommendationService	recommendation_data_access — success	14:35:06
escalationService	escalation_triggered — confidence 55%	14:35:11
notificationService	operator_notification_delivered	14:35:11
governance_engine	hitl_enqueued — awaiting quincy	14:36:02
quincy	hitl_decision — approved	14:36:09

Captured from a real run against the live system — every timestamp, actor, and event name is genuine output, not illustrative copy.

THE GUARDRAIL

"An action gets blocked because it lacks approval — not because the system can't figure out what to do, but because it isn't allowed to decide alone."

One rule outranks everything else in this design

Every proposed remediation passes through a pure, deterministic check before an execution path is even reachable: it must be linked to real incident evidence, of an allowed reversible action type, and carry a genuine human approval record. This isn't a policy comment enforced by convention — it's a function every code path runs through, and it's what a login system exists to make meaningful: the approver on record is the operator who actually signed in, verified by a password hash, not a name string nobody checked.

WHAT'S REAL

Status, by claim — not by intention

Everything below is backed by a test a reviewer can run, or a live run this session actually performed. Gaps are named, not hidden.

Capability	Evidence	Tests	Status
Continuous monitoring	Real polling interval, overlap-safe	12	BUILT

Capability	Evidence	Tests	Status
DMV / cloud read + fixture fallback	Parameterized query, honest source tag	25	BUILT
AI root-cause & confidence	Real Claude calls over live evidence	23	BUILT
Escalation & notification	Real ntfy.sh push delivery	24	BUILT
Remediation guardrail	Evidence + type + approval, pure & deterministic	10	BUILT
Human-approval queue	Assigned-approver check, correlation-linked	16	BUILT
Correlation-ID audit trail	One ID reconstructs a full incident story	24	BUILT
Session-based auth	scrypt-hashed password, httpOnly cookie	19	BUILT
Rollback for reversible actions	Real revert, confirmed via live status check	6	BUILT
Multi-factor authentication	No second factor exists yet	—	HONEST GAP
Live Azure SQL this month	Free-tier quota exhausted, renews Sep 1	—	EXTERNAL

Counts above are the capability's own test file(s). The 210-test total also includes shared infrastructure — retry/timeout handling, static-asset serving, reliability circuit-breaking — that supports every row here but isn't one capability on its own.

STACK

What it's actually built with

LANGUAGE TypeScript end to end	GATEWAY Plain Node http, MCP over stdio
GOVERNANCE Dependency-free, pure functions	CONSOLE React 19 + Vite, same-origin served
DATA Azure SQL Database, Blob Storage	REASONING Anthropic Claude API
NOTIFICATIONS ntfy.sh push delivery	TESTING Vitest — 210 tests, 3 packages

SEE IT RUNNING

The whole stack runs locally against real Azure and Anthropic services — nothing here is mocked for the demo.

```
cd mcp-server && npm install && npm run http  
# open http://localhost:8787/
```

Repository: github.com/qninying/ai-operations-center

CoreOps · Colaberry Enterprise AI Leadership Accelerator